

CSE 620 — Combinatorial Optimization

Project 2: Linear Programming with the Simplex Method

[Team Member 1 Last Name] and [Team Member 2 Last Name]

University of Louisville — CSE 620, Fall 2026

Code: github.com/.../cse_620_combinatorial_optimization_project_2

Video: [SharePoint / Drive / YouTube unlisted link]

October 3, 2026

1 Introduction

1.1 Objective

The objective of this project is to apply the Simplex algorithm to a series of increasingly larger linear programming (LP) problems, and to study how the number of decision variables n and the number of constraints m affect (i) the wall-clock time required to obtain a solution, (ii) the number of pivots taken by the algorithm, and (iii) the nature of the solution itself (optimal, infeasible, unbounded, degenerate, sparse).

We implement or adapt Simplex-based LP software, set up LP instances of increasing size on a prescribed (m, n) grid, record time and pivot counts, and analyze the resulting scaling behavior and the classical failure modes of the method (infeasibility, unboundedness, degeneracy, alternative optima).

1.2 Optimization problem instances

Every instance studied in this project is a linear program in *standard inequality form*

$$\begin{aligned} z^* &= \max_{x \in \mathbb{R}^n} c^\top x \\ \text{s.t. } & Ax \leq b, \quad x \geq 0, \end{aligned} \tag{1}$$

where $c \in \mathbb{R}^n$ is the profit (cost) vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the right-hand side. Some special-case instances also include $\geq$ or $=$ constraints; these are converted to the equality standard form described in Section 2.

Task A — Lecture instance ($n = 4$, $m = 2$)

The representative lecture example used throughout this report is

$$\begin{aligned} \max_x \quad & 3x_1 + 5x_2 + 4x_3 + 2x_4 \\ \text{s.t.} \quad & x_1 + 2x_2 + x_3 + x_4 \leq 12, \\ & 2x_1 + x_2 + 3x_3 + 2x_4 \leq 18, \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned} \tag{2}$$

If the course lecture uses a different $n = 4$, $m = 2$ instance, it is a drop-in replacement in `src/problems.py::lecture_example`.

Tasks B–C — Scaling grid

Following the project specification we solve the grid

- **variables:** $n \in \{4, 10, 20, 30, 40, 50\}$ (from 4 to 50 in steps of 10),
- **constraints:** $m \in \{2, 6, 10, 14\}$ (from 2 to 14 in steps of 4),

with 10 random seeds per (m, n) cell on the *main grid* (240 solves). Main-grid instances are random nonnegative programs

$$A_{ij} \sim \mathcal{U}(0, 1), \quad c_j \sim \mathcal{U}(0, 1), \quad b = A x_{\text{ref}} + \delta, \quad \delta \sim \mathcal{U}(0, 0.5)^m, \quad (3)$$

with a sparse nonnegative reference point x_{ref} . Because $A \geq 0$ and $b \geq 0$, $x = 0$ is feasible and the recession cone is trivial, so every main-grid instance is feasible and bounded; this isolates the effect of (m, n) on pivot count and runtime.

Two additional mixed-constraint grids (signed coefficients, $\leq$ and $\geq$ rows) are used in the analysis to study infeasible and unbounded behavior: a *well-posed* grid (a reference point is strictly feasible by construction) and a *stress* grid (random right-hand-side signs).

Metrics recorded per solve

For each instance we record: status (optimal / infeasible / unbounded), objective value z^* , total pivots (Phase I + Phase II), solve time of the custom Simplex, solve time of SciPy/HiGHS, number of nonzero entries in x^* , sparsity $1 - \text{nnz}(x^*)/n$, whether the optimal vertex is degenerate, and whether alternative optima exist.

2 Methods

2.1 Reference software

The assignment recommends the **Python SciPy library for Linear Programming**:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>

We use that toolbox (`scipy.optimize.linprog` with the HiGHS backend) as the reference Simplex-based LP solver for cross-validation of status and objective values. Because HiGHS does not expose the number of simplex pivots, we also implement a **custom two-phase tableau Simplex** (`src/simplex.py`) that counts every pivot and reports the diagnostics required by the project. All experiments were run on a laptop (CPU only) under Python 3.9 with NumPy 2.0, SciPy 1.13, and Matplotlib 3.9.

2.2 Standard form

Problem (1) is converted to equality standard form by introducing one slack variable per $\leq$ constraint:

$$Ax + s = b, \quad x \geq 0, \quad s \geq 0. \quad (4)$$

Rows with $b_i < 0$ are multiplied by -1 (the sense flips accordingly). $\geq$ rows receive a surplus variable $u \geq 0$ and an artificial variable $a \geq 0$; $=$ rows receive an artificial variable. Artificials are needed only for Phase I and are removed before Phase II. In matrix form the Phase I system is

$$\max w = -\mathbf{1}^\top a \quad \text{s.t.} \quad \tilde{A}x + Su + Ia = \tilde{b}, \quad x, u, a \geq 0, \quad (5)$$

where $\tilde{A}, \tilde{b}$ are the sign-flipped data.

2.3 Simplex pivoting — equations

Let B be the current basis matrix and N the nonbasic columns. The basic solution and objective value are

$$x_B = B^{-1}b - B^{-1}Nx_N, \quad z = c_B^\top B^{-1}b + \sum_{j \in N} \bar{c}_j x_j, \quad (6)$$

with *reduced costs*

$$\bar{c}_j = c_j - c_B^\top B^{-1}A_j. \quad (7)$$

Algorithm 1 Two-phase tableau Simplex (maximization)

```
1: Convert to standard form (4); add surplus/artificial variables; build initial basis  $B = I$ 
2: Phase I: set  $c_j^I = -1$  for artificial columns, 0 otherwise
3: while some  $\bar{c}_j^I > 0$  do
4:    $e \leftarrow \arg \max_j \bar{c}_j^I$  ▷ Dantzig
5:   if all  $(B^{-1}A_e)_i \leq 0$  then return infeasible
6:   end if
7:    $r \leftarrow \arg \min_{i: (B^{-1}A_e)_i > 0} (B^{-1}b)_i / (B^{-1}A_e)_i$ 
8:   pivot on  $(r, e)$ ; update tableau and basis
9: end while
10: if  $w^* < -\varepsilon$  then return infeasible ▷  $\sum a_i > 0$ 
11: end if
12: drive artificials out of the basis (drop redundant rows)
13: Phase II: restore true costs  $c$ ; rebuild objective row
14: while some  $\bar{c}_j > 0$  do
15:    $e \leftarrow \arg \max_j \bar{c}_j$ 
16:   if all  $(B^{-1}A_e)_i \leq 0$  then return unbounded
17:   end if
18:   ratio test; pivot on  $(r, e)$ 
19: end while
20: return optimal  $x^*$ ,  $z^* = c^\top x^*$ 
```

- **Optimality (maximization):** the current basis is optimal iff $\bar{c}_j \leq 0$ for all nonbasic j .
- **Entering variable:** choose $e = \arg \max_j \bar{c}_j$ (Dantzig rule); if $\bar{c}_e \leq 0$ stop.
- **Ratio test / leaving variable:**

$$\theta^* = \min_{i: (B^{-1}A_e)_i > 0} \frac{(B^{-1}b)_i}{(B^{-1}A_e)_i}, \quad r = \arg \min_{i: (B^{-1}A_e)_i > 0} \frac{(B^{-1}b)_i}{(B^{-1}A_e)_i}. \quad (8)$$

- **Pivot:** replace column e by column r in the basis and update the tableau by Gaussian elimination.
- **Unboundedness:** if no row has $(B^{-1}A_e)_i > 0$, the LP is unbounded (the column defines a feasible improving ray).

To avoid cycling on degenerate vertices the implementation uses Dantzig's rule and automatically switches to **Bland's rule** (smallest index with $\bar{c}_j > 0$; among tied ratio-test rows, the smallest basic index) after a safeguard number of pivots.

2.4 Two-phase algorithm — pseudocode

2.5 Code, platform, and usage

Custom solver. `src/simplex.py` implements `simplex_solve(c, A_ub, b_ub, A_eq, b_eq, sense)` with the same argument convention as SciPy, plus a `sense` flag in `{'max','min'}`. The returned `SimplexResult` contains status, x , objective, pivots (total / Phase I / Phase II), solve time, degeneracy and alternative-optima flags, nonzero count, and sparsity.

Reference solver. SciPy is called as `linprog(-c, A_ub=A, b_ub=b, bounds=[(0,None)]*n, method='highs')` for a maximization problem (SciPy always minimizes), and the objective sign is flipped back. HiGHS reports `nit` (iterations) and runtime but *not* simplex pivot counts, which is why the custom solver is required for requirement C.

Experiment drivers.

- `scripts/run_lecture_example.py` — Task A
- `scripts/run_grid_experiment.py` — Tasks B–C (main grid, mixed grids, special cases)

Table 1: Task A: lecture LP solved by the custom two-phase Simplex and by SciPy/HiGHS.

	Custom Simplex	SciPy HiGHS
Status	optimal	optimal
Objective z^*	37.200	37.200
x^*	(0, 3.6, 4.8, 0)	(0, 3.6, 4.8, 0)
Pivots / iterations	2 (P1: 0, P2: 2)	nit = 2
Solve time	0.16–0.26 ms	0.54–0.77 ms
Sparsity of x^*	50% (nnz = 2)	50%
Degenerate / multi-optima	no / no	—

Table 2: Main grid at $m = 2$: mean $\pm$ std over 10 seeds. Times in milliseconds.

n	Pivots	Simplex (ms)	SciPy (ms)	z^*	Sparsity
4	1.9 ± 0.7	0.241	1.191	2.46	0.625
10	2.5 ± 0.7	0.140	0.532	10.07	0.810
20	3.4 ± 0.7	0.167	0.575	21.00	0.910
30	3.2 ± 1.0	0.157	0.552	47.58	0.937
40	4.2 ± 1.2	0.178	0.567	49.44	0.950
50	4.0 ± 0.7	0.193	0.650	84.87	0.960

- analysis/analyze_results.py — Task D (tables, plots)

Listing 1: Custom Simplex and SciPy/HiGHS on the lecture LP (Task A).

Minimal usage examples.

```
import numpy as np
from src.simplex import simplex_solve
from scipy.optimize import linprog

c = np.array([3.0, 5.0, 4.0, 2.0])
A = np.array([[1.0, 2.0, 1.0, 1.0],
              [2.0, 1.0, 3.0, 2.0]])
b = np.array([12.0, 18.0])

res = simplex_solve(c, A_ub=A, b_ub=b, sense="max")
print(res.status, res.objective, res.x, res.pivots)
# optimal 37.2 [0. 3.6 4.8 0.] 2

sc = linprog(-c, A_ub=A, b_ub=b,
             bounds=[(0, None)] * 4, method="highs")
print(sc.status, -sc.fun, sc.x, sc.nit)
# 0 37.2 [0. 3.6 4.8 0.] 2
```

3 Results

3.1 Task A — Lecture instance ($n = 4$, $m = 2$)

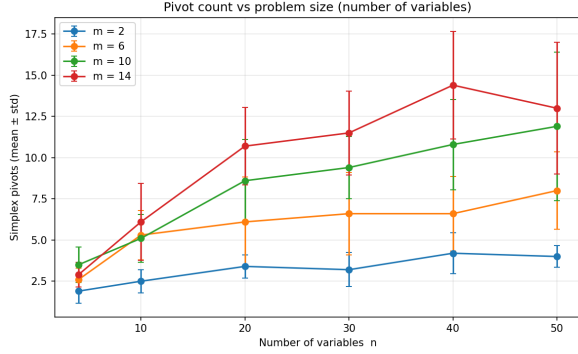
Table 1 reports the solve of (2). The custom Simplex reaches the optimum $z^* = 37.2$ at $x^* = (0, 3.6, 4.8, 0)$ in **2 pivots** (Phase I: 0, Phase II: 2) and 0.16–0.26 ms. Both constraints are tight at the optimum (slacks = 0), and SciPy/HiGHS returns the same objective and solution.

3.2 Tasks B–C — Scaling on the (m, n) grid

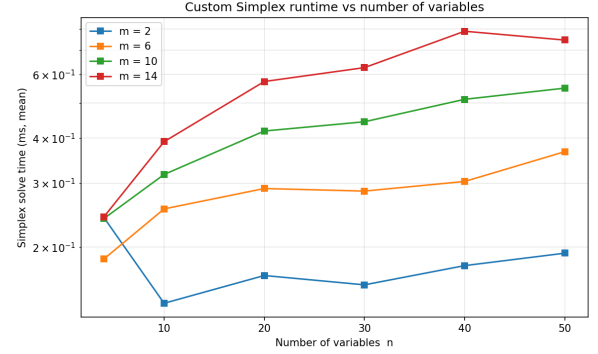
Table 2 gives the full scaling study at $m = 2$ as n grows from 4 to 50, and Table 3 gives the mean pivot count over the entire (m, n) grid (10 seeds per cell, all 240 solves optimal, 100% agreement with SciPy on the objective). Figures 1a and 1b plot pivots and runtime versus n .

Table 3: Mean number of Simplex pivots over 10 seeds, for each (m, n) cell of the main grid. Mean runtimes (ms) are reported in Table 5 in the appendix.

Vars n	Constraints m			
	2	6	10	14
4	1.9	2.6	3.5	2.9
10	2.5	5.3	5.1	6.1
20	3.4	6.1	8.6	10.7
30	3.2	6.6	9.4	11.5
40	4.2	6.6	10.8	14.4
50	4.0	8.0	11.9	13.0



(a) Pivots vs. n for each m .



(b) Custom Simplex time vs. n (log scale).

Figure 1: Scaling of pivot count and runtime with the number of variables n on the main grid.

3.3 Special-case instances (phenomena)

Table 4 lists the hand-crafted instances that exhibit each behavior expected by the project: infeasibility, unboundedness, degeneracy, alternative optima, and sparse optimal solutions. Figure 2 visualizes pivots, time, and sparsity for these cases; the Phase I/Phase II breakdown appears in Appendix A.

3.4 Mixed-constraint grids

On the mixed $\leq/\geq$ grids (signed coefficients) the solver no longer returns only optima. Over 120 solves per grid:

- **Well-posed mode** (reference point strictly feasible): 29 optimal, 91 unbounded.
- **Stress mode** (random RHS signs): 5 optimal, 20 infeasible, 95 unbounded.

Cross-check agreement with SciPy is 100% on both grids (status and objective). Heatmaps of the infeasible/unbounded fractions are in Appendix A.

4 Experimental Analysis

4.1 Effect of the number of variables n

Fixing $m = 2$ (Table 2, Figure 1) the mean pivot count grows slowly but steadily from 1.9 ($n = 4$) to 4.0 ($n = 50$) — a factor of only $\sim 2\times$ for a $12.5\times$ increase in n . Runtime stays in the 0.14–0.24 ms band: at these sizes the dense tableau pivot costs $O(mn)$, and the overhead of the Python loop dominates the arithmetic. SciPy/HiGHS is consistently slower in wall-clock time (0.53–1.19 ms at $m = 2$) because of per-call setup overhead, even though its internal `nit` is comparable to our pivot count.

The *optimal objective* grows with n (e.g. $z^* \approx 84.9$ at $n = 50$, $m = 2$) simply because more nonnegative variables with positive profit coefficients are available to pack into the same resource budget.

Table 4: Special-case LPs solved by the custom Simplex (SciPy agrees on every status and optimal objective).

Instance	m	n	Status	z^*	Pivots	De-gen	Multi
Lecture (Task A)	2	4	optimal	37.200	2	—	—
Infeasible parallel	2	2	infeasible	—	1 (P1)	—	—
Unbounded ray $d = (1, 1)$	2	2	unbounded	—	1 (P2)	—	—
Degenerate vertex	3	2	optimal	18.000	2	yes	—
Multiple-optima face	3	2	optimal	4.000	1	yes	yes
Sparse optimum	4	20	optimal	1.043	1	—	—
Sparse optimum	6	50	optimal	1.294	1	—	—
Bounded base	2	2	optimal	4.000	1	—	yes
After removing constraint	1	2	unbounded	—	0	—	—

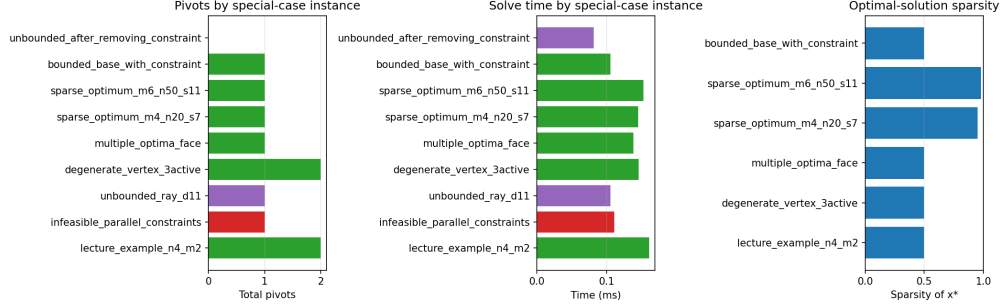


Figure 2: Special-case instances: pivots (left), solve time (middle), and sparsity of x^* (right). Colors follow solver status.

4.2 Effect of the number of constraints m

Table 3 shows that m is the stronger driver of pivot count. At $n = 50$ the mean pivots rise from 4.0 ($m = 2$) to 8.0 ($m = 6$), 11.9 ($m = 10$) and 13.0 ($m = 14$) — more than a $3\times$ increase. At $n = 20$ the same trend holds ($3.4 \rightarrow 6.1 \rightarrow 8.6 \rightarrow 10.7$). Runtime tracks the same pattern ($0.19 \rightarrow 0.37 \rightarrow 0.55 \rightarrow 0.75$ ms at $n = 50$; full matrix in Table 5).

Why? Each additional constraint cuts the feasible polyhedron and can create new vertices; the Simplex walks from vertex to vertex along edges, so a richer vertex set lengthens typical pivot paths. Increasing n at fixed m mostly adds candidate directions (columns) that the ratio test filters out quickly, whereas increasing m changes the geometry of the polyhedron itself. This is the classical contrast between “wide” and “tall” LPs: pivot counts and runtime scale more with the number of constraints than with the number of variables in these dense random instances.

4.3 Time versus pivots

Comparing the left and right panels of Table 3 and Table 5, runtime is nearly proportional to the mean pivot count at fixed (m, n) cell size (e.g. $n = 50$: 4.0 pivots $\rightarrow$ 0.19 ms vs. 13.0 pivots $\rightarrow$ 0.75 ms), as expected for a dense tableau method in which each pivot is $O(mn)$ Gaussian elimination. Deviations at very small n reflect timer resolution and interpreter overhead rather than algorithmic effects.

4.4 When Simplex fails to yield a solution

Infeasibility. Phase I maximizes $w = -\sum_i a_i$. A strictly negative optimum $w^* < 0$ means the artificial variables cannot all be driven to zero, i.e. the constraint set is empty. The special case $x_1 + x_2 \leq 4$, $x_1 + x_2 \geq 6$ terminates in Phase I after 1 pivot with $w^* = -2$. On the stress grid, 20 of 120 instances are infeasible — typically when $\geq$ rows demand more of a shared resource than $\leq$ rows allow. This is not a failure of the algorithm: it is the algorithm *correctly* certifying that no feasible point exists.

Unboundedness. If a nonbasic column e has $\bar{c}_e > 0$ but $(B^{-1}A_e)_i \leq 0$ for all rows i , the ray $x(t) = x_B + tB^{-1}A_e$ ($t \geq 0$) stays feasible while $z(t) = z + t\bar{c}_e \rightarrow \infty$. The special case $\max x_1$ s.t. $x_1 - x_2 \leq 2$ exhibits the recession direction $d = (1, 1)$ with $Ad \leq 0$ and $c^T d = 1 > 0$. More instructively, taking the bounded program $\max x_1 + x_2$ s.t. $x_1 + x_2 \leq 4$, $x_2 \leq 3$ ($z^* = 4$) and *removing* the first constraint leaves only $x_2 \leq 3$; the ray $(t, 3)$, $t \rightarrow \infty$, is then feasible and improving, so the LP becomes unbounded with 0 pivots. Removing constraints enlarges the feasible region and can destroy boundedness — a practical warning when editing models.

4.5 Degeneracy and multiple optima

A vertex is *degenerate* when more than n constraints are active, so at least one basic variable equals zero. The instance $\max 3x_1 + 9x_2$ s.t. $x_1 + 4x_2 \leq 8$, $x_1 + 2x_2 \leq 4$, $x_2 \leq 2$ has optimum $(0, 2)$ where all three constraints are tight; the solver reports the degenerate flag and still terminates (Bland safeguard). Alternative optima occur when some nonbasic variable has $\bar{c}_j = 0$ at optimality: for $\max x_1 + x_2$ s.t. $x_1 + x_2 \leq 4$, $x_1 \leq 4$, $x_2 \leq 4$ every point on the face $x_1 + x_2 = 4$ is optimal, and the solver correctly flags `multiple_optima = true`.

On the *main* random grid the degeneracy and multi-optima fractions are 0.00 in every cell: with continuous random coefficients, degenerate vertices and zero reduced costs occur with probability zero. Degeneracy is a real phenomenon in structured models (tight budgets, rounded data, redundant constraints), which is why it is studied through the hand-crafted cases rather than random instances.

4.6 Sparsity of optimal solutions

Table 2 shows sparsity rising from 0.625 ($n = 4$, $m = 2$) to 0.960 ($n = 50$, $m = 2$); at $m = 6$ it reaches 0.898 at $n = 50$, and the dedicated sparse packing instances reach 0.95–0.98. This is structural, not accidental: at any basic feasible solution at most m of the n decision variables are nonzero (the basic variables), so

$$\text{nnz}(x^*) \leq m \implies \text{sparsity} \geq 1 - \frac{m}{n}. \quad (9)$$

For $m = 2$, $n = 50$ this lower bound is 96%, matching the observed mean. In applications — production planning (only a few products manufactured), portfolio corner solutions (few assets held), network flows (few links used) — the zeros in x^* are the operational insight: the optimal vertex activates only a handful of activities. Interior-point methods, in contrast, return analytic-center-like solutions with many nonzero entries.

4.7 Custom Simplex versus SciPy/HiGHS

Every one of the 240 main-grid solves and every special case agrees with SciPy on status and objective (to 10^{-6} relative tolerance). The custom tableau implementation is competitive in wall-clock time at these sizes and is the only instrument that reports true simplex pivot counts. HiGHS may internally use dual simplex, primal simplex, or an interior-point crossover depending on the model, so its `nit` is not directly comparable to our pivot log — a methodological caveat worth stating in the report.

5 Conclusion

We implemented a two-phase tableau Simplex with pivot counting, validated it against the recommended SciPy/HiGHS toolbox on 240+ LP instances, and studied how m and n affect solution time, pivot count, and the structure of the answer.

Key findings.

1. **Constraints dominate variables.** Mean pivots grew from ~ 4 to ~ 13 as m went from 2 to 14 at $n = 50$ ($3.25\times$), but only from ~ 2 to ~ 4 as n went from 4 to 50 at $m = 2$ ($2.1\times$). Runtime followed pivots almost linearly, as expected for dense tableau updates.
2. **Simplex is fast on generic LPs.** All 240 main-grid instances solved in well under 1 ms with 2–20 pivots; random nondegenerate instances rarely approach the worst-case Klee–Minty behavior.

3. **Failure modes are diagnosable.** Phase I residual artificials certify infeasibility; a positive reduced cost with no positive column entry certifies unboundedness. Removing a bounding constraint converted a bounded LP ($z^* = 4$) into an unbounded one with zero pivots.
4. **Optimal vertices are sparse.** $\text{nnz}(x^*) \leq m$ guarantees sparsity $\geq 1 - m/n$; observed sparsity matched this bound (e.g. 96% at $m = 2$, $n = 50$).
5. **Degeneracy is nongeneric but real.** Random instances showed 0% degeneracy, while structured instances triggered degeneracy and alternative optima — handled correctly by the Bland safeguard.

What we learned. The project made concrete the textbook picture of Simplex: a walk on vertices whose length is governed mainly by the *constraint* geometry, with failure modes that the two-phase construction detects rather than hides. Cross-validating a from-scratch implementation against a production solver (SciPy/HiGHS) is an effective way to gain confidence in both.

Possible improvements. (a) Use the *revised* Simplex method with sparse B^{-1} updates to scale beyond $n = 50$; (b) compare against an interior-point method to quantify the vertex versus interior trade-off; (c) generate structured instances (e.g. Klee–Minty cubes) to exhibit worst-case pivot counts explicitly; (d) time repeated solves to separate warm-start effects from cold-start overhead; (e) add dual-feasibility / network-simplex variants for the sparse application models discussed above.

A Additional Figures and Tables

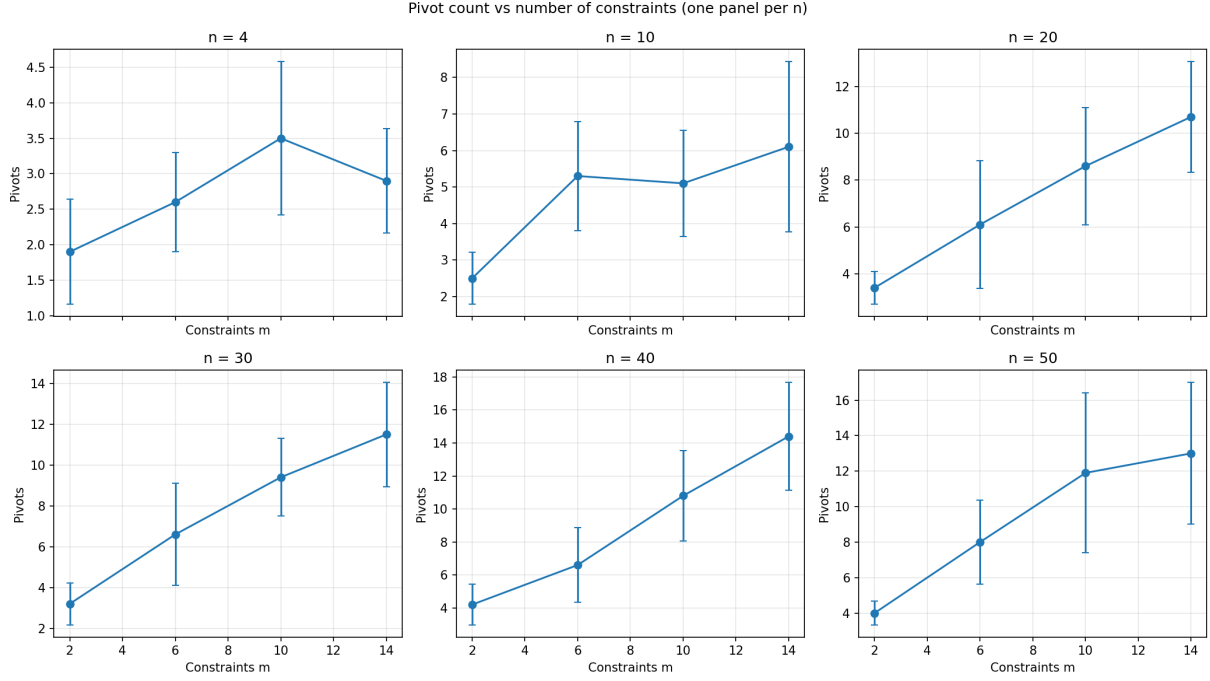


Figure 3: Mean pivots versus m , one panel per n .

Table 5: Mean Simplex runtime (ms) over 10 seeds, for each (m, n) cell of the main grid.

Vars n	Constraints m			
	2	6	10	14
4	0.241	0.186	0.240	0.243
10	0.140	0.255	0.318	0.391
20	0.167	0.290	0.419	0.574
30	0.157	0.286	0.444	0.627
40	0.178	0.304	0.512	0.790
50	0.193	0.367	0.550	0.747

B Code Excerpts

B.1 Core pivot step (src/simplex.py)

```
# Objective row stores [-reduced_costs | z] so that Gaussian
# elimination on the full tableau keeps T[-1,-1] = z.
def _make_tableau(c_vec, A, b, bas):
    m = A.shape[0]
    T = np.zeros((m + 1, A.shape[1] + 1))
    T[:-1, :-1] = A
    T[:-1, -1] = b
    c_B = c_vec[list(bas)]
    T[-1, :-1] = c_B @ A - c_vec    # = -reduced costs
    T[-1, -1] = float(c_B @ b)      # z
    return T

# One simplex phase: enter on most negative objective-row entry,
# leave by the minimum ratio test; detect unboundedness.
def _run_phase(T, bas, c_vec, max_pivots):
    pivots = 0
```

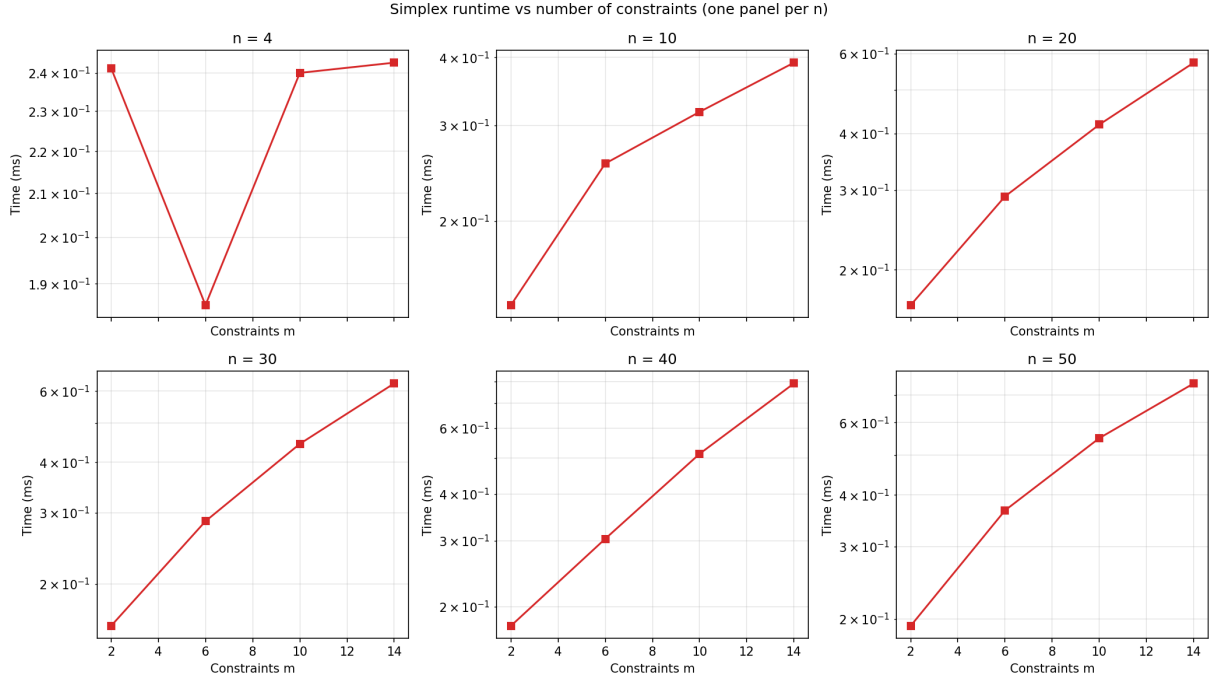


Figure 4: Mean Simplex runtime versus m , one panel per n .

Table 6: Main grid: mean optimal objective z^* over 10 seeds.

Vars n	$m = 2$	$m = 6$	$m = 10$	$m = 14$
4	2.456	1.307	1.151	1.035
10	10.071	3.970	3.473	3.454
20	21.004	8.993	7.502	6.858
30	47.576	12.934	10.609	11.203
40	49.440	20.229	16.261	14.271
50	84.868	24.415	20.246	19.159

```

use_bland = False
while True:
    obj_row = T[-1, :-1]
    if np.all(obj_row >= -TOL):           # optimal
        return "optimal", pivots, bas
    if not use_bland and pivots >= BLAND_SWITCH_PIVOTS:
        use_bland = True                # anti-cycling safeguard
    e = int(np.argmax(obj_row))          # entering column
    col = T[-1, e]
    pos = np.nonzero(col > TOL)[0]
    if pos.size == 0:                   # improving ray exists
        return "unbounded", pivots, bas
    ratios = T[:-1, -1][pos] / col[pos]
    r = int(pos[int(np.argmin(ratios))]) # leaving row
    piv_val = T[r, e]
    T[r, :] /= piv_val
    for i in range(T.shape[0]):
        if i != r:
            T[i, :] -= T[i, e] * T[r, :]
    bas[r] = e
    pivots += 1

```

B.2 Experiment driver (src/experiment.py)

```

from src.problems import random_feasible_lp
from src.experiment import run_grid, N_VALUES, M_VALUES

```

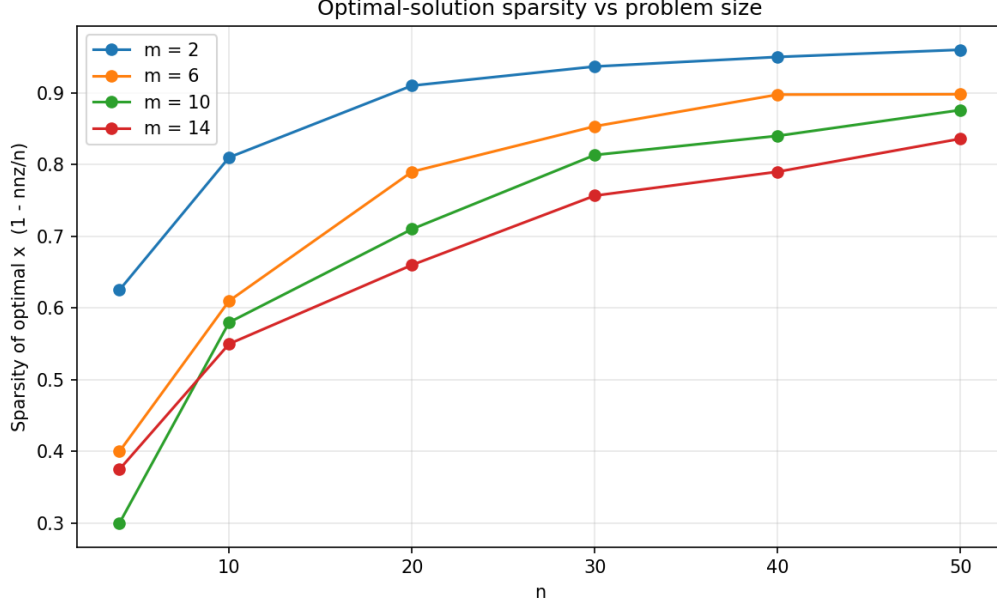


Figure 5: Sparsity of x^* versus n for each m .

Table 7: Main grid: SciPy HiGHS nit (mean) and SciPy runtime (ms, mean) for reference.

n	nit				SciPy ms			
	2	6	10	14	2	6	10	14
4	2.0	3.4	6.3	5.3	1.191	0.572	0.569	0.533
10	2.8	5.0	8.8	9.3	0.532	0.642	0.625	0.611
20	3.5	7.3	10.2	14.2	0.575	0.655	0.648	0.657
30	2.7	7.8	12.7	14.5	0.552	0.618	0.645	0.739
40	3.1	8.9	12.8	15.7	0.567	0.754	0.835	0.971
50	3.3	10.5	13.6	16.9	0.650	0.735	0.793	0.998

```
# Tasks B+C: n = 4..50 step 10, m = 2..14 step 4, 10 seeds each
df = run_grid(N_VALUES, M_VALUES, n_seeds=10,
             generator="random_feasible", use_scipy=True)
# columns: status, objective, pivots, simplex_time, scipy_time,
#          sparsity, degenerate, multiple_optima, ...
df.to_csv("results/grid_main.csv", index=False)
```

B.3 Package layout

```
src/simplex.py      # two-phase tableau Simplex + SciPy cross-check
src/problems.py    # LPInstance, generators, special cases
src/experiment.py  # grid runners, metrics
scripts/run_lecture_example.py # Task A
scripts/run_grid_experiment.py # Tasks B+C
analysis/analyze_results.py   # Task D tables & plots
simplex_project.ipynb          # interactive walkthrough
results/                      # CSVs, tables/, plots/, ANALYSIS.md
```

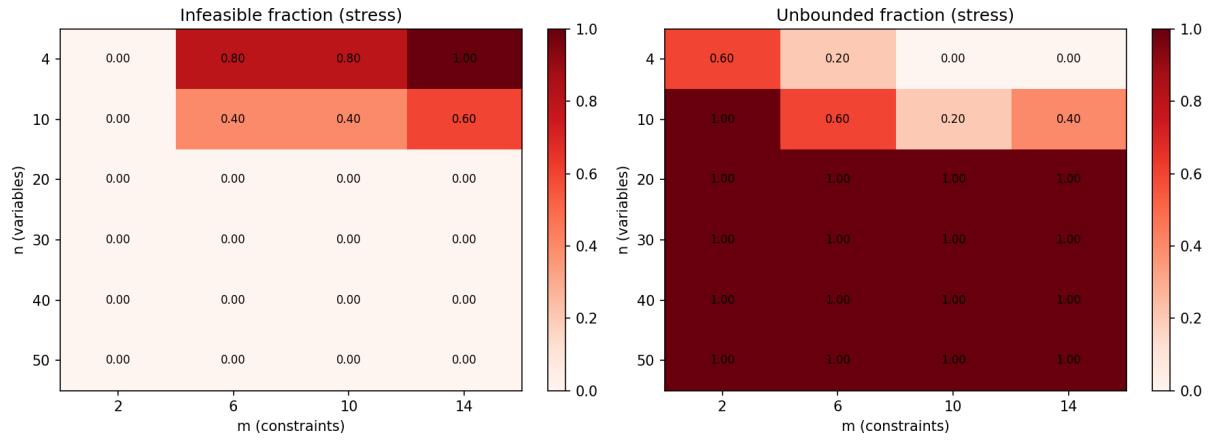


Figure 6: Infeasible and unbounded fractions on the mixed-constraint stress grid.

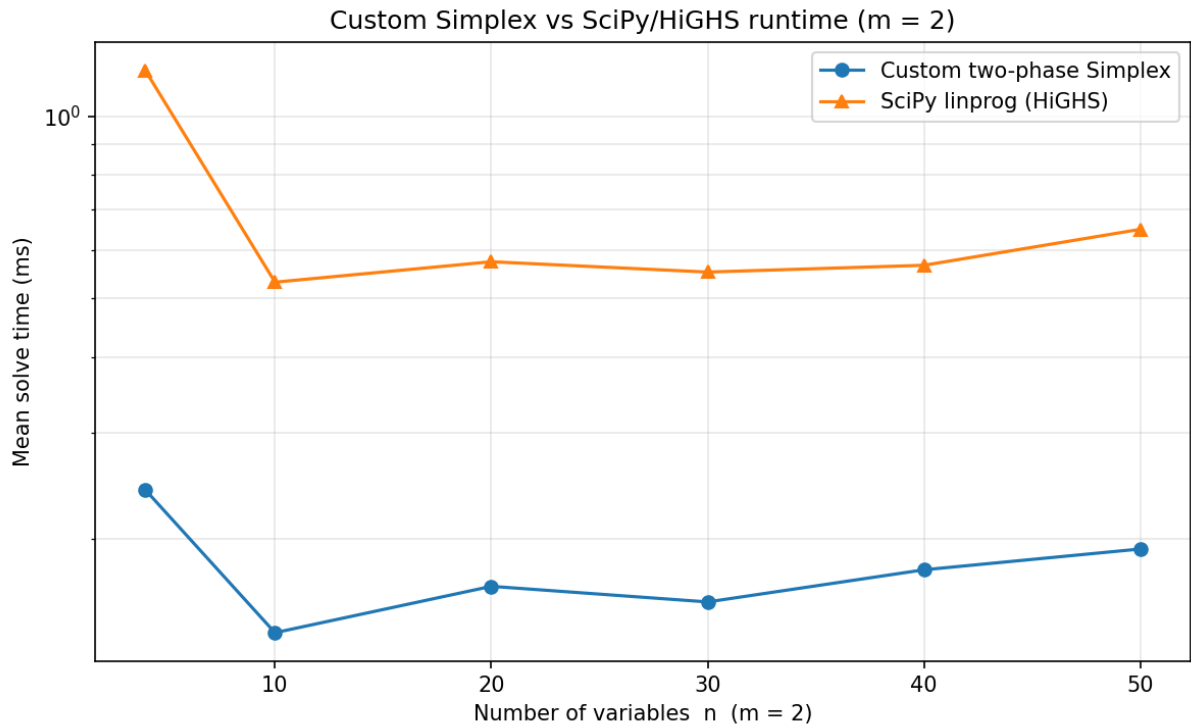


Figure 7: Custom Simplex versus SciPy/HiGHS runtime at $m = 2$.

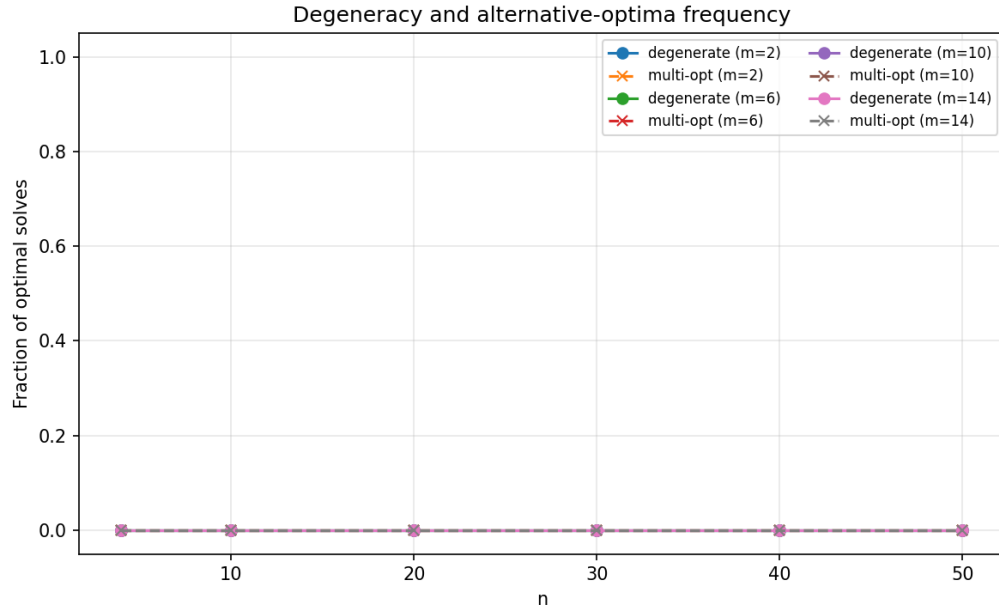


Figure 8: Degeneracy and alternative-optima frequency on the main grid (both ≈ 0 for random continuous data).

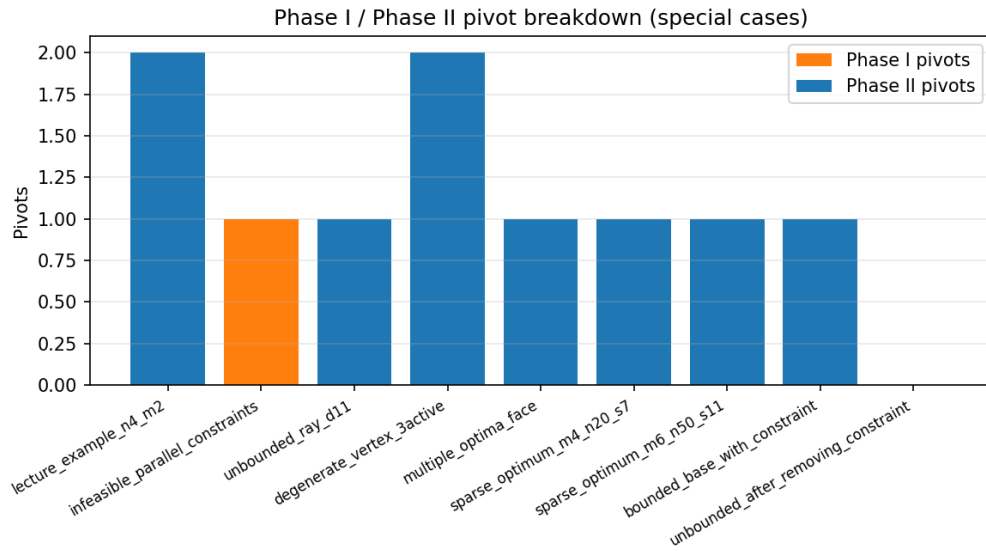


Figure 9: Phase I / Phase II pivot breakdown for the special cases.